Assignment 3: Parallel Matrix Multiplication in Java

Thread Scaling of a Row-Parallel i-k-j Kernel on a 4+6-Core Apple M4, with SIMD Vectorisation

Máximo Prosperi Reynoso
BIG DATA, GCID, ULPGC
Individual Assignment 3, 2026–2027
https://github.com/Xami650/Individual-Assignments-Big-Data-ULPGC

4 November 2026

Abstract

This study asks how the number of worker threads changes the performance of a shared-memory matrix multiplication in Java, and when parallel execution helps. The validated i-k-j kernel of Assignment 2 is the sequential baseline. The parallel version computes the rows of C with the same loop on a fixed pool of $p = 1$ to 20 worker threads, with rows claimed one at a time (dynamic) or assigned in contiguous bands (static), for $n = 64$ to 4096 on an Apple M4 with 4 performance (P) and 6 efficiency (E) cores. All 167 configurations gave bitwise-identical results. Parallel execution helped from $n = 128$ on. With the 4 P cores the speedup was 2.3 to 2.8, and with all 10 cores it was 3.0 to 4.3 for $n \geq 256$, an efficiency of 0.30 to 0.43. At $n = 64$ a multiplication lasts 30 μs: the best speedup was 1.15, and with one thread the parallel code was 24% slower than the sequential one. Per-process counters, including the CPU time spent on P cores, explain why 10 threads were not 10 times faster. macOS ran four threads on the P cores and the others on E cores, which retired 0.46 to 0.64 times the instructions per second of a P core. The clock fell from 4.0 GHz with one busy core to 3.0 to 3.8 GHz on the P cores and 2.8 to 2.9 GHz on the E cores. Contention for the shared caches and memory raised the cycles per multiply-add by 11 to 50% with four threads. At $n = 4096$, from $p = 4$ on, the kernel requested the matrix B at about the 120 GB/s memory bandwidth of the chip, and the speedup stayed between 2.4 and 3.6. Oversubscription (12 to 20 threads) cost at most 5% and gained clearly only at $n = 1024$. Against the hypothesis, static bands were up to 32% faster than dynamic rows at $n = 256$ and 1024. The most likely cause is that adjacent rows, computed at the same time by different threads, share cache lines. The JIT's SIMD vectorisation made the kernel 1.16 to 1.51 times faster with 40 to 49% fewer instructions, and it gained less with more threads. Peak memory grew with p only at small n, from task objects allocated on every call. The conclusions hold for the tested kernel and machine only.

1 Introduction and objective

Assignment 2 made the sequential Java kernel 3 to 10 times faster by reordering its loops to i-k-j, so that the innermost loop adds a multiple of a row of B to a row of C. That kernel still used one core, while the machine of these studies has ten: four performance (P) cores and six efficiency (E) cores. This study keeps the validated i-k-j kernel, computes the rows of C on several threads, and asks how the number of threads changes its performance.

Research question. *How does the number of worker threads (1 to 20) affect the runtime, speedup, efficiency and memory of a row-parallel version of the validated i-k-j matrix multiplication in Java, for $n = 64$ to 4096 on a 10-core Apple M4, and when does parallel execution help?* The factors are the thread count p and the matrix size n, with the way rows are assigned to threads

(the schedule) as a secondary factor. The outcomes are the time per multiplication, the speedup $S(p) = T_{seq}(n)/T_p(n)$, the efficiency $E(p) = S(p)/p$, the overhead of the parallel code with one thread, the peak memory of the process, and process counters that explain the times: CPU time, the share of it spent on E cores, instructions and cycles. T_{seq} is the time of the sequential baseline and T_p the time of the parallel method with p threads on the same inputs; both are medians. Thread counts above 10, the number of logical processors, are *oversubscribed*.

Vectorisation (optional extension). *How much does the JIT's automatic SIMD vectorisation contribute at a fixed thread count, and does its effect change with the number of threads?* The same code is run with the vectoriser of the JIT compiler switched on (the default) and off.

Hypotheses. The hypotheses below were fixed in the protocol file (`config/bench.json`) before the reported session. They are predictions, and Section 5 tests them against the results.

- H1 The parallel method is bitwise identical to the sequential one for every thread count and schedule, with or without vectorisation.
- H2 With one thread, the parallel method pays a fixed cost per call for handing the work to a pool thread: above 10% at $n = 64$, below 1% from $n = 512$.
- H3 For $n \leq 128$ a multiplication is too short to amortise waking and joining p threads: the speedup stays below 2 and can fall below 1.
- H4 The gain per added thread is largest up to $p = 4$, the number of P cores; threads beyond 4 run partly on E cores and add less, so $S(10)$ is well below 10. With a static assignment of rows the slowest core sets the time, so the static schedule is slower than the dynamic one for $4 < p \leq 10$.
- H5 The kernel reads all of B for every row of C, so several cores compete for the shared cache and memory bandwidth: the cycles per multiply-add, summed over all cores, rise with p, more so for $n \geq 2048$, where B no longer fits in the L2 cache. No superlinear speedup is expected.
- H6 Oversubscription gives no further speedup and, with the dynamic schedule, costs little (within 5% of $p = 10$).
- H7 Peak memory does not depend on p beyond a few MiB.
- H8 Without vectorisation the compiled inner loop is scalar, the instructions per multiply-add roughly double, and the kernel slows down most where it is limited by execution ($p = 1$, $n \leq 1024$) and least where it is limited by memory traffic ($p = 10$, $n = 2048$).

2 Background and references

Speedup and its limits. If a fraction f of a computation cannot be parallelised, p processors give at most $S(p) = 1/((1 - f) + f/p)$, Amdahl's bound [1]. In a parallel matrix multiplication with fixed work per row, the serial part is the time to start and join the threads, and the idle time at the end of a call when the rows run out unevenly. On a chip whose cores differ in speed, p threads do not add up to p cores of work either: Hill and Marty extend Amdahl's model to such asymmetric chips [2]. A kernel can also be limited by memory traffic rather than by arithmetic. The roofline model bounds the attainable rate by the memory bandwidth times the operational intensity, the floating-point operations per byte moved from memory [3]. When several cores share a cache or a memory channel, the bandwidth per core falls as cores are added. The clock rate also varies: chips run their cores faster when few are active and slow them down as power and temperature rise [4].

Assigning rows to threads. In the i-k-j order, the rows of C are independent: row i depends on row i of A and on all of B, which are only read. The rows can be split among threads in two classic ways, named after OpenMP's `schedule` clause [5]. A *static* schedule fixes the assignment in advance, for example one contiguous band of n/p rows per thread, and costs nothing at run time. A *dynamic* schedule lets each thread claim the next unclaimed chunk of rows when it becomes free, which costs a shared counter but adapts to threads that run at different speeds. On Apple silicon, Apple's tuning guide warns that "if you use a static distribution, threads running on higher efficiency cores will finish much sooner than threads running on lower efficiency cores" [6].

Java threads and shared data. The Java memory model defines when a write by one thread is visible to a read by another: only if the write *happens-before* the read (JLS §17.4.5 [7]). The classes of `java.util.concurrent` extend this to their operations: actions before the submission of a task to an `ExecutorService` happen-before the task runs, and "actions taken by the asynchronous computation represented by a `Future` happen-before actions subsequent to the retrieval of the result via `Future.get()` in another thread" [8]. A fixed thread pool keeps its threads alive between calls, so a call pays for waking them, not for creating them [9].

Cores and scheduling on macOS. The Apple M4 has four P cores, which share a 16 MiB L2 cache, and six E cores, which share a 4 MiB L2 cache. macOS decides on which core type a thread runs; an application influences this only through the quality-of-service (QoS) class of its work, and background work tends to run on the E cores [6]. There is no way to bind a thread to a core: on the machine of this study, the affinity interface of the kernel (`thread_policy_set` with `THREAD_AFFINITY_POLICY`) returns `KERN_NOT_SUPPORTED`. macOS does account the CPU time, instructions and cycles of a process separately for the P cores (`struct rusage_info_v6`, returned by `proc_pid_rusage` [10]), which this study uses to see where its threads ran.

SIMD vectorisation. HotSpot's optimising JIT compiler (C2) packs independent iterations of simple loops into SIMD instructions with its SuperWord pass, after the superword-level parallelism method of Larsen and Amarasinghe [11]. On the M4 the vectors are 128-bit NEON registers, which hold two doubles. The pass is on by default and is switched off with `-XX:-UseSuperWord`. Java forbids contracting $a \cdot b + c$ into a fused multiply-add [12], so vector and scalar code perform the same rounded operations. Assignment 2 inferred vectorisation from instruction counts only.

Measuring Java code. The JIT compiles a method while it runs, so the first executions run different code from the steady state, and benchmarks must discard warm-up iterations [14]. Assignment 2 found that a single long call could stay in on-stack-replacement code unless the method was first compiled on small inputs, and added a pre-warm for this reason.

3 Methodology

3.1 Implementations

All code is in `src/main/java/org/ulpgc/bd/matmul/` and was written for this study, except the baseline, which is copied unchanged from Assignment 2. No library is used in any measured method. Both methods compute in IEEE-754 double precision on the same `double[][]` storage (an array of row arrays) and the same inputs, and both perform the same i-k-j loop for every row (Fig. 1).

- **ikj** (sequential baseline, `DenseMatMul.ikj`): the i-k-j kernel of Assignment 2 (tag `assignment2-v1.0`), validated there against NumPy and bitwise against the Assignment 1 baseline. It runs on the main thread and no thread pool exists.

```
// row i of C (ikjRow; ikj loops it over i)
double[] ai = a[i], ci = c[i];
Arrays.fill(ci, 0.0);
for (int k = 0; k < n; k++) {
  double aik = ai[k]; double[] bk = b[k];
  for (int j = 0; j < n; j++)
    ci[j] += aik * bk[j];
}
```

```
// par_ikj: one task per worker thread
// dynamic
for (int i = next.getAndIncrement(); i < n;
     i = next.getAndIncrement())
  ikjRow(a, b, c, n, i);
// static, worker t of p
for (int i = t*n/p; i < (t+1)*n/p; i++)
  ikjRow(a, b, c, n, i);
// caller: pool.invokeAll(tasks), then get()
```

Figure 1: The measured code (abbreviated). `next` is a fresh `AtomicInteger` per call; the bounds of the static bands use 64-bit arithmetic.

- **par_ikj** (parallel, `ParallelMatMul.ikj`): the rows of C are distributed among p worker threads, and each row is computed by `ikjRow`, the loop body of ikj for one i. The workers belong to a `ThreadPoolExecutor` with exactly p threads, created and started before any timing. A call builds p tasks, hands them to the pool with `invokeAll` and waits; the calling thread never computes a row, so p counts the threads that compute. Two schedules assign the rows:
  - *dynamic* (the main configuration): each worker claims the next unclaimed row from a shared `AtomicInteger`, one row at a time, until none are left;
  - *static*: worker t computes the contiguous band of rows from $\lfloor tn/p \rfloor$ to $\lfloor (t+1)n/p \rfloor - 1$.

Shared data. A and B are only read, and each row of C is written by exactly one task. The inputs exist before the tasks are submitted, and `invokeAll` returns only after every task has completed; the happens-before guarantees of the executor and of `Future.get()` (Section 2) make every write of a worker visible to the caller. Rows are separate arrays, so two threads can touch the same 128-byte cache line only at the boundary between two row arrays. Every entry C_{ij} receives the same operations in the same order as in ikj, so the results should be bitwise identical (H1).

3.2 Inputs

The inputs are those of Assignments 1 and 2, generated by the same unchanged code: for each n, A and then B are drawn uniformly on $[-1, 1)$ from NumPy's PCG64 generator [17] seeded with `SeedSequence([2026, n])`, and stored as raw little-endian float64. All 24 files used here (12 sizes) have the same SHA-256 digests as the Assignment 2 files, which are tracked in `data/checksums.sha256`.

3.3 Experiments

Table 1 lists the three experiments of the reported session, 167 configurations in all, each run in its own JVM process. Thread counts from 1 to 10 locate the changes at $p = 4$ (the P cores) and $p = 10$ (all logical processors). Counts of 12, 16 and 20 are oversubscribed. The sizes bracket the caches. B takes 32 KiB at $n = 64$ and 2 MiB at $n = 512$, which fits both L2 caches. At 1024 it takes 8 MiB, which fits only the 16 MiB L2 of the P cores, and at 2048 and 4096 it takes 32 and 128 MiB, which fit neither. In every size block, the sequential ikj and the parallel method with 10 threads also run at the start and at the end as drift controls (Section 3.7).

3.4 Execution environment

Table 2 lists the machine and the software. These settings were the same for every run.

Table 1: Experiments of the reported session (167 configurations). The SuperWord-on counterparts of V_simd are the P_threads configurations with the same method, n and p.

Experiment	Methods	Sizes and threads	Configs
P_threads	ikj; par_ikj, dynamic	$n = 64$, 128, 256, 512, 1024, 2048, 4096; $p = 1$ to 10, 12, 16, 20	119[a]
P_schedule	par_ikj, static	$n = 256$, 1024, 2048; same p	39
V_simd	`-XX:-UseSuperWord`: ikj; par_ikj, dynamic, $p = 4$ and 10	$n = 512$, 1024, 2048	9

[a] 91 par_ikj configurations and 28 drift controls (ikj and par_ikj with $p = 10$, first and last in each of the 7 size blocks).

Table 2: Execution environment and fixed settings of the reported session (`20261104-030623`, recorded in `env.txt` before and after the session).

Machine	MacBook Air 15-inch, M4, 2025 (Mac16,13), fanless; 24 GiB RAM with 120 GB/s memory bandwidth [16]; AC power, Low Power Mode off
CPU	Apple M4: 4 P cores (L1d 128 KiB each, 16 MiB L2 shared) and 6 E cores (L1d 64 KiB each, 4 MiB L2 shared); 10 physical cores and 10 logical processors (no SMT); 128 B cache lines; 16 KiB pages
Affinity	none: `thread_policy_set` with `THREAD_AFFINITY_POLICY` returns `KERN_NOT_SUPPORTED` (46); the scheduler places the threads, and the P-core share of CPU time is measured
OS	macOS 26.6.2 (build 25G83), Darwin 25.6.0
Java	OpenJDK 25.0.1 (build 25.0.1+8-27, HotSpot 64-bit Server VM), Maven 3.9.12; `java -Xmx8g`; defaults otherwise: G1 collector (4 MiB regions, 9 parallel and 2 concurrent GC threads), tiered compilation with 4 compiler threads, SuperWord on, 16-byte maximum vector size, 2 MiB thread stacks
Threads	with ikj the process has 6 Java threads (21 operating-system threads, including the JIT and GC threads); par_ikj adds its p workers (16 and 31 at $p = 10$, 26 and 49 at $p = 20$)
Python	CPython 3.13.9 (Anaconda build) with NumPy 2.3.5, pandas 2.3.3, matplotlib 3.10.6 and psutil 7.0.0, only for inputs, checks, the harness and the analysis
Source	commit `1736461`; only the drafts of the report, the README and the analysis script were uncommitted
Conditions	43.1 min starting at 03:06 on 4 November 2026, with the desktop applications and a container virtual machine closed; no thermal or performance warning before or after; 1-minute load average 1.27 at the start (just after the build); other processes used a median of 0.06 cores during the runs

3.5 Process counters and resource metrics

Around every sample, the driver reads the counters of its own process through the Java foreign function interface [13]: `proc_pid_rusage` with `RUSAGE_INFO_V6` gives the CPU time (user plus system), the part of it on P cores, the instructions and the cycles, each in total and on P cores; `proc_pidinfo` gives the context switches and the number of threads of the process [10]. All are sums over the threads of the process. From the differences over the measured samples the analysis derives, per configuration: the busy cores U (CPU time divided by elapsed time), the share of CPU time on E cores, the mean clock rate of the busy cores f (cycles divided by CPU time), and the cycles and instructions per multiply-add. Ratios of sums make the identity $T = C/(fU)$ exact for the mean time per multiplication T and the cycles per multiplication C, so the speedup of the means splits into three factors,

$$\bar{S} = \frac{C_{seq}}{C_p} \cdot \frac{f_p}{f_{seq}} \cdot \frac{U_p}{U_{seq}},$$

extra cycles (contention), clock rate (frequency and core type) and idle threads. The driver also records the garbage collections, the JIT compilation time and the bytes allocated on the Java heap during the measured samples, the number of Java and operating-system threads, and the CPU time of the process per QoS class. Around every process, the harness reads the system-wide CPU time (`psutil`) and subtracts the process's own CPU time, which gives the CPU time used by other processes during the run.

Memory is measured as in Assignments 1 and 2: the peak resident set size (RSS) and the peak memory footprint of the whole process, from `/usr/bin/time -l`, next to the element storage of the three matrices, $24n^2$ bytes.

3.6 Correctness checks

The Java code has 32 JUnit 5 tests. They cover hand-computed 2×2 and 3×3 products, identity and zero matrices, an exact BigDecimal reference, the overwriting of a reused output, and par_ikj bitwise against ikj for $n \in \{1, 2, 3, 7, 33, 64, 65, 100\}$, $p \in \{1, 2, 3, 4, 7, 10, 16, 20\}$ (so also $p > n$) and both schedules. Further tests make 200 repeated calls on one pool, to expose races between calls, and check the static bands, the propagation of an exception thrown in a worker, the rejection of $p < 1$, the process counters, the file format and the driver's output contract.

Before benchmarking, `scripts/verify.py` ran 15 configurations through their normal command line: ikj; par_ikj with $p \in \{1, 2, 3, 4, 10, 16\}$ and both schedules; and ikj and par_ikj ($p = 4$) with SuperWord off. The cases are those of Assignments 1 and 2: $n \in \{1, 2, 3, 10, 64, 100, 256\}$ with random inputs (reference `numpy.matmul`), AI (exact answer A) and $A \cdot 0$ (exact answer 0). A result passes if $|C_{ij} - R_{ij}| \leq 2\gamma_n(|A||B|)_{ij}$ for every entry, with $\gamma_n = nu/(1 - nu)$ and $u = 2^{-53}$. This is the tolerance of Assignment 1: it bounds the difference between two results computed in any summation order [15]. In addition, all configurations of a case must agree bit for bit (H1). During the benchmark, every sample prints the sum of the entries of C. This consumes the result, and it is compared across all configurations of each n.

3.7 Measurement protocol

The protocol is that of Assignment 2, with drift controls and cool-down pauses added for the parallel runs.

Timer boundary. Only the multiplication is timed, with `System.nanoTime()`. For ikj this is the kernel call, including the clearing of each row of C. For par_ikj it is the whole `ParallelMatMul.ikj` call: building the p tasks, handing them to the workers, which are already running, the computation, and the wait until every task has finished. Creating the pool and starting its threads, loading the inputs, starting the JVM and the pre-warm are outside the

timed region. The time of par_ikj with one thread therefore includes the cost of handing the work to another thread, which is the overhead the assignment asks for.

Pre-warm, warm-up and samples. Each configuration runs in one fresh JVM. The JVM creates its pool and then runs the method for 1 s on small random inputs ($n = \min(N, 256)$) with the same pool, so that the JIT compiles the kernel with a complete profile. Warm-up then continues until at least one sample and 1 s of warm-up work have run. Each sample times k back-to-back multiplications, with $k = \max(1, \lceil 0.1\,\mathrm{s}/t \rceil)$ and t the time per multiplication of the latest warm-up. Five measured samples follow. The summaries report the median and the interquartile range (IQR) of the time per multiplication over these five samples. Warm-up samples are kept in the raw data and excluded from every summary.

Speedup. $S(p)$ is the median time of the first ikj run of the same size block divided by the median time of the configuration, and $E(p) = S(p)/p$. Its interval, shown as error bars, is $[Q_1^{seq}/Q_3^p,\ Q_3^{seq}/Q_1^p]$.

Run order, drift controls and cool-down. The session runs one block per size, in ascending n. Each block starts with ikj and par_ikj (dynamic, $p = 10$) and ends with the same two in reverse order. The other configurations of that size run in between, in the order given by `random.Random(2026 + n).shuffle` over the declared list, so that the thread count is not confounded with the time within the block. The last/first ratios of the two controls measure the drift of the machine over the block, for one core and for all cores. After each process the harness pauses for as long as the process ran, at least 5 s and at most 90 s.

Pilot. A pilot session, not part of the reported data ($n = 2048$ and 4096, two samples per configuration, 2 s pauses), fixed the session length. It also showed thermal throttling. In the multi-threaded runs at $n = 4096$, the mean clock of the busy cores (cycles per CPU second) fell from about 3.1 GHz at the start of the block to about 2.0 GHz at its end, while single-thread runs stayed at 4.0 GHz; the MacBook Air has no fan. A drift control with one thread cannot see this. The cool-down pauses and the drift control with all cores were therefore added and declared in the protocol before the reported session.

Budgets. Each process had 60 s to load its inputs, create the pool and pre-warm, and then at most 120 s for every sample. The heap was limited to 8 GiB (`-Xmx8g`) and the session to 90 min. A failure would have been recorded as a result; none occurred.

Background load. Around every process, the harness reads the system-wide busy CPU time (`psutil`) and subtracts the CPU time of the process, which gives the CPU time used by other processes during the run. In the reported session, `/usr/bin/time` printed its times with a decimal comma, because the shell ran under a Spanish locale, and the harness did not parse them. The analysis therefore takes the process's CPU time from the CPU times per QoS class that the driver reads from `proc_pid_rusage` at exit, which cover the whole process. In the pilot and a smoke session, which recorded both, the two agreed within 0.02 s per process. The parser was fixed after the session.

3.8 Vectorisation evidence

Two kinds of evidence show what the vectoriser does. The first is the machine code. `scripts/simd_evidence.py` runs the driver once with SuperWord on and once with it off, for ikj and for par_ikj ($p = 4$, dynamic, $n = 512$), with HotSpot's diagnostic `-XX:CompileCommand=print`. These flags are used only in this separate run, never in a timed

one. Without the `hsdis` disassembler plugin, HotSpot prints the compiled code as hexadecimal instruction words. The script turns them back into instructions with `clang` (as `.inst` directives) and `objdump`. In the final C2-compiled code of each kernel method, it then counts the two-lane vector (`fmul.2d`, `fadd.2d`) and the scalar double-precision multiplies and adds, the fused multiply-adds, and the 128-bit (`q` register) loads and stores. The files were regenerated after the session with the session's JDK. The counts equal those of the files produced on the day of the pilot, when the default JDK was OpenJDK 25.0.2. The second kind of evidence is the counters: the instructions and cycles per multiply-add of the V_simd configurations and of their SuperWord-on counterparts.

4 Results

4.1 The session and its controls

The session ran 167 configurations in 43.1 min, and all completed with five measured samples. The relative IQR was 0.42% in the median configuration and at most 10.7% (static, $p = 12$, $n = 2048$). It exceeded 2% in 20 configurations: 14 at $n \geq 2048$ and 6 static runs at $n = 256$ and 1024. Other processes used a median of 0.06 cores during the runs (90th percentile 0.18, maximum 1.29 in a single run). One garbage collection, shorter than 1 ms, fell into a measured sample (par_ikj, $p = 4$, $n = 64$). The JIT compiler threads were still active during the measured samples of every configuration, for 2 to 25 ms of compilation time (median 4 ms). That is at most about 5% of the measured time, and it ran on a compiler thread, not in the timed kernel.

The drift controls (Table 5, last two columns) stayed within ±1.2% for ikj except at $n = 128$. There, the last run was 7.4% faster than the first, with the same instructions per multiply-add and the same 4.04 GHz clock but fewer cycles (0.540 against 0.583 per multiply-add). The difference lies between two JVMs running the same code, and every speedup at $n = 128$ depends on the choice of baseline run by up to 7.4%. For $p = 10$, the ratio was 0.996 to 1.033, except at $n = 2048$ (0.875). The first $p = 10$ run at that size took 380 ms, against 330 and 332 ms for the shuffled and the last run. It used 11% more cycles per multiply-add for the same instructions while other processes used 0.40 cores; the speedups use the shuffled run. At $n = 4096$ the last $p = 10$ run, at the end of a block of about 20 min, ran its cores at 2.59 GHz instead of 3.03 GHz, a sign of thermal throttling. It still took only 2.5% longer, because its cycles per multiply-add fell by 12%.

4.2 Correctness

All 32 unit tests passed; they were run again with the session's JDK. `verify.py` checked 315 (case, configuration) results. All were within the tolerance (largest error 0.13 of the bound), and in every case all 15 configurations were bitwise identical. In the benchmark session, the checksums of every configuration agreed exactly for each n (167 configurations, 835 measured samples).

4.3 Time, speedup and efficiency

Table 3 gives the median time per multiplication, and Table 4 the speedups and efficiencies. Fig. 2 plots them against p, and Fig. 3 against n. With four threads, the number of P cores, the speedup was 2.3 to 2.8 for every $n \geq 128$. Threads beyond four raised it more slowly, to 3.0 to 4.3 at $p = 10$ for $n \geq 256$. The efficiency fell with every added thread, from 0.58 to 0.69 at $p = 4$ to 0.30 to 0.43 at $p = 10$ ($n \geq 256$). The largest speedup with at most 10 threads was 4.27 ($n = 1024$); with 16 threads at the same size it was 4.86. Against n, the speedup with 10 or more threads rose to its maximum at $n = 1024$ and fell at 2048 and 4096. With four threads it stayed between 2.3 and 2.8 from $n = 128$ on.

At $n = 64$ a sequential multiplication took 30.0 μs. No thread count made it more than 1.15 times faster (at $p = 6$), and with one to four threads the parallel method was slower than ikj ($S = 0.81$ to 0.98). With 10 threads the process kept only 2.9 cores busy on average, and it made

Table 3: Median time per multiplication (ms) of ikj and of par_ikj (dynamic) for each p and n. Last row: the largest relative IQR among the configurations of the column.

p	$n = 64$	$n = 128$	$n = 256$	$n = 512$	$n = 1024$	$n = 2048$	$n = 4096$
ikj (seq.)	0.0300	0.301	2.23	17.4	138	1 173	12 982
1	0.0372	0.288	2.22	17.4	138	1 196	13 253
2	0.0312	0.209	1.50	12.2	80.7	846	8 090
3	0.0306	0.130	1.19	8.87	62.5	618	5 917
4	0.0306	0.115	0.889	7.29	50.5	503	4 686
5	0.0296	0.121	0.852	6.55	46.4	454	5 354
6	0.0261	0.121	0.813	5.92	41.7	399	4 258
7	0.0276	0.121	0.773	5.33	38.6	370	4 332
8	0.0271	0.121	0.743	5.01	36.1	339	3 574
9	0.0274	0.120	0.707	4.64	33.4	353	4 316
10	0.0275	0.120	0.664	4.38	32.4	329	4 366
12†	0.0285	0.120	0.665	4.37	32.1	337	4 568
16†	0.0271	0.121	0.667	4.37	28.4	341	4 566
20†	0.0290	0.113	0.667	4.37	28.8	342	4 589
largest IQR	0.6%	0.8%	0.6%	0.7%	1.4%	8.9%	4.4%

† oversubscribed: more worker threads than the 10 logical processors.

Table 4: Speedup $S(p)$ and, in parentheses, efficiency $E(p)$ of par_ikj (dynamic) over the first ikj run of each size block.

p	$n = 64$	$n = 128$	$n = 256$	$n = 512$	$n = 1024$	$n = 2048$	$n = 4096$
1	0.806 (0.81)	1.04 (1.04)	1.00 (1.00)	0.998 (1.00)	1.00 (1.00)	0.981 (0.98)	0.980 (0.98)
2	0.962 (0.48)	1.44 (0.72)	1.49 (0.74)	1.43 (0.71)	1.71 (0.86)	1.39 (0.69)	1.60 (0.80)
3	0.979 (0.33)	2.31 (0.77)	1.87 (0.62)	1.96 (0.65)	2.21 (0.74)	1.90 (0.63)	2.19 (0.73)
4	0.979 (0.24)	2.62 (0.65)	2.50 (0.63)	2.39 (0.60)	2.74 (0.68)	2.33 (0.58)	2.77 (0.69)
5	1.01 (0.20)	2.49 (0.50)	2.61 (0.52)	2.66 (0.53)	2.98 (0.60)	2.59 (0.52)	2.42 (0.48)
6	1.15 (0.19)	2.48 (0.41)	2.74 (0.46)	2.94 (0.49)	3.31 (0.55)	2.94 (0.49)	3.05 (0.51)
7	1.09 (0.16)	2.48 (0.35)	2.88 (0.41)	3.27 (0.47)	3.58 (0.51)	3.17 (0.45)	3.00 (0.43)
8	1.10 (0.14)	2.49 (0.31)	3.00 (0.37)	3.47 (0.43)	3.83 (0.48)	3.46 (0.43)	3.63 (0.45)
9	1.09 (0.12)	2.50 (0.28)	3.15 (0.35)	3.75 (0.42)	4.14 (0.46)	3.32 (0.37)	3.01 (0.33)
10	1.09 (0.11)	2.51 (0.25)	3.35 (0.34)	3.98 (0.40)	4.27 (0.43)	3.56 (0.36)	2.97 (0.30)
12†	1.05 (0.09)	2.50 (0.21)	3.35 (0.28)	3.98 (0.33)	4.31 (0.36)	3.48 (0.29)	2.84 (0.24)
16†	1.10 (0.07)	2.48 (0.15)	3.34 (0.21)	3.99 (0.25)	4.86 (0.30)	3.44 (0.21)	2.84 (0.18)
20†	1.03 (0.05)	2.66 (0.13)	3.34 (0.17)	3.98 (0.20)	4.79 (0.24)	3.43 (0.17)	2.83 (0.14)

† oversubscribed: more worker threads than the 10 logical processors.

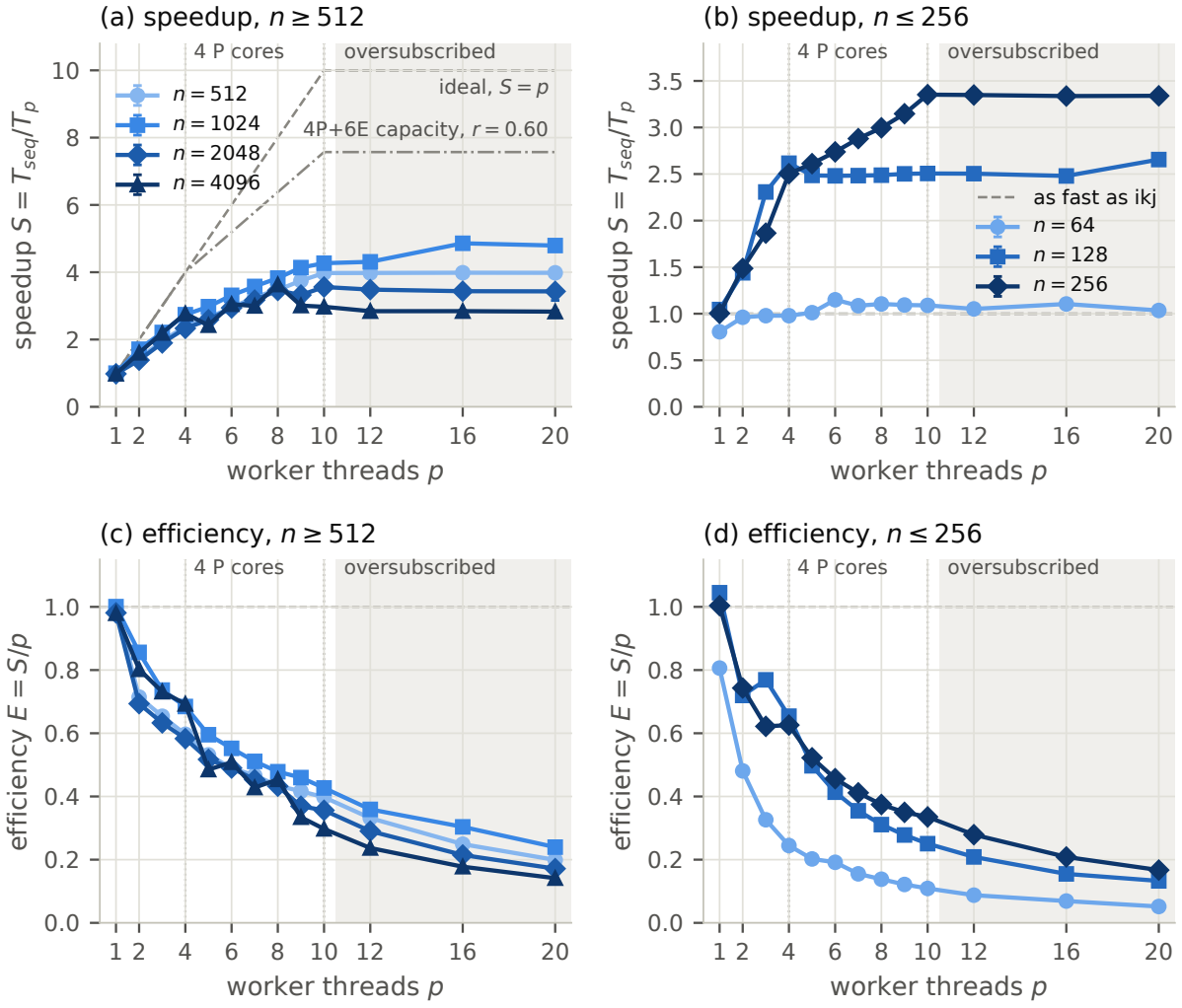


Figure 2: Speedup (top) and efficiency (bottom) of par_ikj (dynamic) against the number of worker threads, for $n \geq 512$ (left) and $n \leq 256$ (right). Error bars: $[Q_1^{seq}/Q_3^p, Q_3^{seq}/Q_1^p]$. Dotted lines: $p = 4$ (P cores) and $p = 10$ (logical processors); shaded: oversubscribed. The 4P+6E capacity line, $\min(p, 4) + r\,\mathrm{clip}(p-4, 0, 6)$, counts an E core as $r = 0.60$ P cores, the ratio of instructions per CPU second on E and P cores measured at $p = 10$, $n = 512$. At $n = 128$, $E(1) > 1$ because the first ikj run, the baseline, was 7.4% slower than the last (Section 4.1).

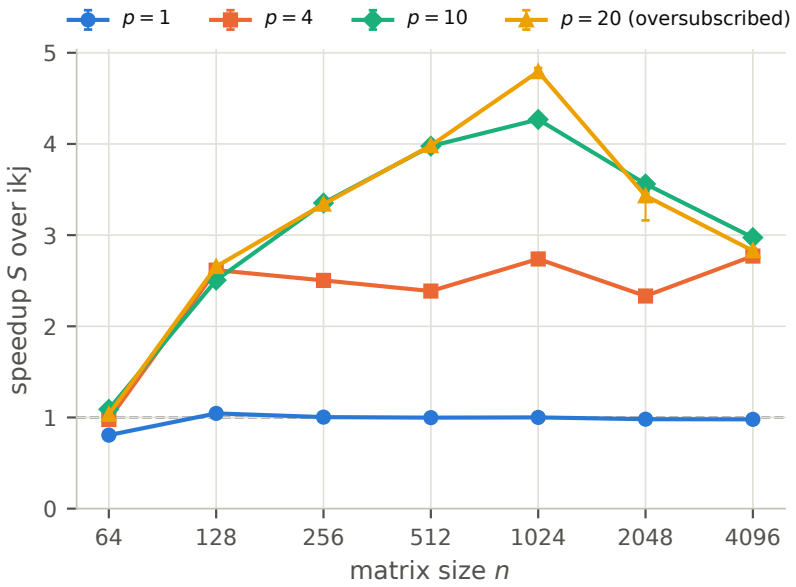


Figure 3: Speedup over ikj against n for $p = 1$, 4, 10 and 20 (dynamic), with the same interval bars as Fig. 2.

Table 5: One thread, the best thread count and the drift, per n. overhead: median time of par_ikj (dynamic, $p = 1$) relative to ikj; static: the same for the static schedule (P_schedule sizes only); best p: the fastest par_ikj configuration (dynamic), with its S and E; drift: median time of the last over the first control run of the block, for ikj and for par_ikj with $p = 10$.

n	ikj (ms)	par, $p = 1$ (ms)	overhead (%)	static (%)	best p	S	E	drift, ikj	drift, $p = 10$
64	0.0300	0.0372	+24.0	–	6	1.15	0.19	1.003	0.996
128	0.301	0.288	-4.3	–	20†	2.66	0.13	0.926	1.000
256	2.23	2.22	-0.4	-0.3	10	3.35	0.34	0.998	1.002
512	17.4	17.4	+0.2	–	16†	3.99	0.25	1.001	1.003
1024	138	138	-0.1	+0.1	16†	4.86	0.30	1.000	1.033
2048	1 173	1 196	+1.9	+0.9	10	3.56	0.36	0.998	0.875
4096	12 982	13 253	+2.1	–	8	3.63	0.45	1.012	1.025

† oversubscribed: more worker threads than the 10 logical processors.

226 000 context switches per second, against about 2 000 at $n \geq 1024$. At $n = 128$ (0.30 ms) the speedup reached 2.6 at $p = 4$ and stayed between 2.5 and 2.7 for every larger p. The busy cores levelled off at 4.2 to 4.5, and the E cores did 7 to 9% of the work.

4.4 One thread: the cost of the hand-off

With one thread, par_ikj was 24.0% slower than ikj at $n = 64$, which is 7.2 μs per call for handing the work to a worker and waiting for it (Table 5). From $n = 256$ to 1024 the difference was within $\pm 0.4\%$. At $n = 128$ it was -4.3% against the first ikj run and $+3.4\%$ against the last, so it is smaller than the 7.4% between the two baseline runs. At $n = 2048$ and 4096 the one-thread version was 1.9 and 2.1% slower than the first ikj run (2.1 and 0.8% against the last). A cost of microseconds per call cannot explain this. The counters show the same instructions per multiply-add (within 0.02%) at the same 4.04 GHz, but 1.9 to 2.0% more cycles. The static schedule with one thread behaved the same way ($+0.9\%$ at $n = 2048$).

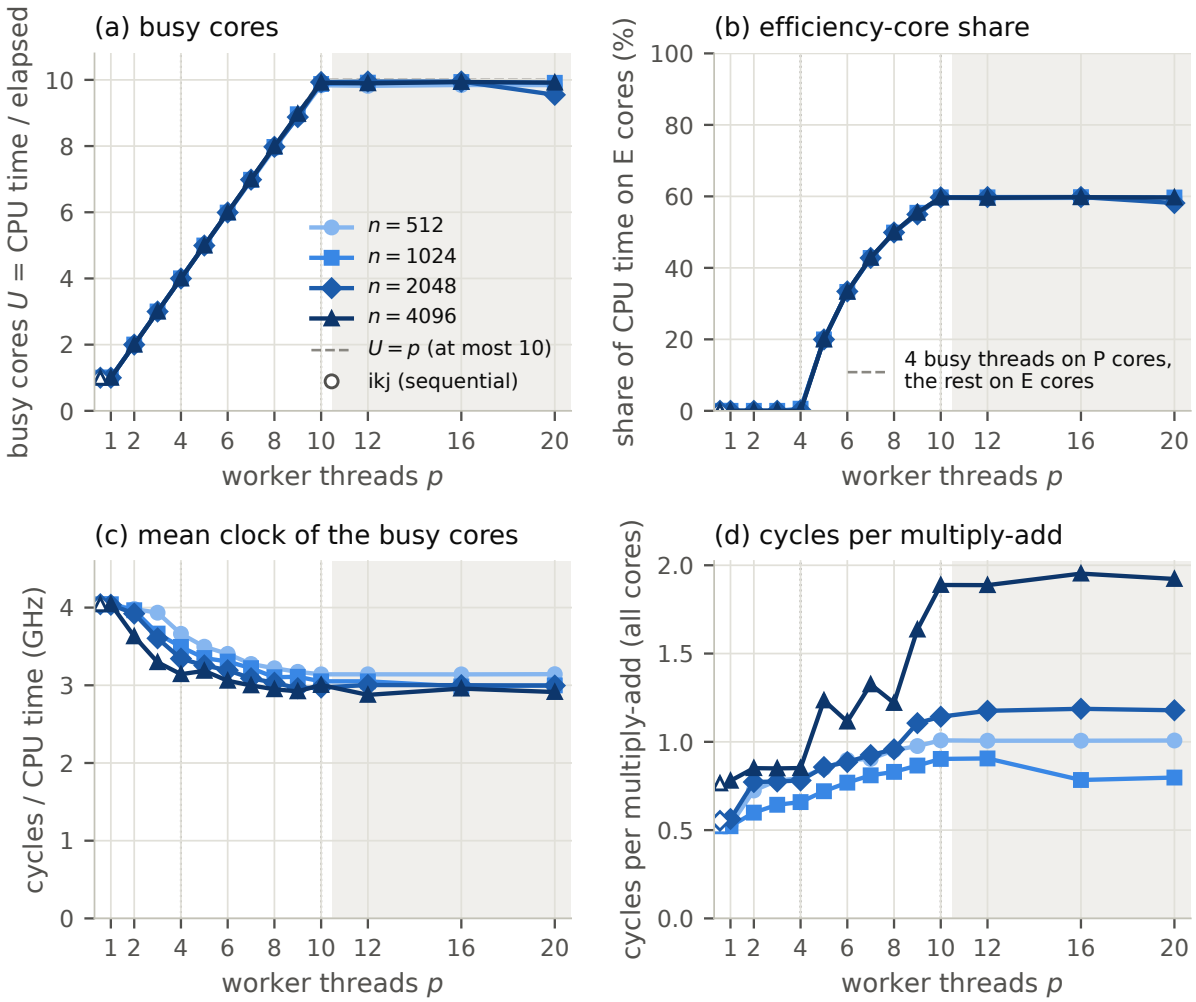


Figure 4: Process counters of par_ikj (dynamic) for $n \geq 512$; hollow markers at the left: ikj. (a) Busy cores, CPU time divided by elapsed time. (b) Share of the CPU time on E cores, with the share expected if four busy threads run on P cores and the others on E cores. (c) Mean clock of the busy cores, cycles divided by CPU time. (d) Cycles per multiply-add, summed over all cores.

4.5 Where the threads ran and what limited them

Fig. 4 shows how the threads used the cores. For $n \geq 256$, the busy cores U followed p up to 10 (9.3 to 9.9 at $p = 10$), so the workers were rarely idle. The share of CPU time on E cores was at most 1% up to $p = 4$. From there it followed $(p - 4)/p$ closely: 20, 33, 43, 50, 55 and 60% for $p = 5$ to 10 at $n = 1024$. macOS therefore kept four threads on the P cores and ran every further thread on an E core. Beyond 10 threads nothing changed: the busy cores and the E-core share stayed at their $p = 10$ values. At $p = 10$, an E core retired 0.46 to 0.64 times as many instructions per CPU second as a P core ($n \geq 256$; 0.60 at $n = 512$, the r of Fig. 2). The clock fell as threads were added. With one busy thread the P cores ran at 4.04 GHz, with four at 3.1 to 3.7 GHz, and the E cores ran at 2.8 to 2.9 GHz.

Table 6 splits the mean speedup into its three factors. At $p = 10$ and $n = 512$ to 4096, 9.8 to 9.9 busy cores gave speedups of only 3.0 to 4.3. Each multiplication needed 1.7 to 2.5 times the cycles of the sequential run ($C_s/C_p = 0.40$ to 0.58), and those cycles ran at 0.74 to 0.78 times the sequential clock. Part of the extra cycles comes from the E cores, which need more cycles per instruction. With four threads, however, all on P cores and with the same instructions as ikj, the cycles per multiply-add were already 1.11 to 1.50 times the sequential ones ($n \geq 256$). This is contention between the P cores for their shared L2 cache and for memory.

Table 6: Decomposition of the speedup of the means, $\bar{S} = (C_s/C_p)(f_p/f_s)(U_p/U_s)$, for $n \geq 512$ and selected p (dynamic). busy U: CPU time over elapsed time; E-core: share of the CPU time on E cores; clock: cycles over CPU time; cycles and instructions per multiply-add are summed over all cores; seq.: ikj.

n	p	$\bar{S}$	busy U	E-core (%)	clock (GHz)	cyc./m.-add	instr./m.-add	C_s/C_p	f_p/f_s	U_p/U_s
512	1	0.998	1.01	0	4.04	0.529	3.21	0.999	1.000	1.000
	2	1.43	2.00	0	3.98	0.725	3.21	0.729	0.985	1.99
	4	2.39	4.00	0	3.66	0.795	3.21	0.665	0.906	3.97
	6	2.94	5.97	33	3.40	0.897	3.20	0.589	0.842	5.92
	8	3.47	7.93	50	3.22	0.953	3.52	0.555	0.796	7.87
	10	3.97	9.84	60	3.14	1.01	3.52	0.524	0.777	9.76
	16†	3.98	9.85	60	3.14	1.01	3.52	0.525	0.777	9.77
	seq.	1	1.01	0	4.04	0.528	3.21	1	1	1
1024	1	1.00	1.00	0	4.04	0.522	3.04	0.999	1.00	1.00
	2	1.71	2.00	0	3.97	0.599	3.04	0.871	0.981	2.00
	4	2.73	4.00	1	3.49	0.659	3.04	0.792	0.864	3.99
	6	3.31	5.99	33	3.31	0.769	3.04	0.678	0.818	5.97
	8	3.84	7.97	50	3.10	0.830	3.45	0.629	0.768	7.95
	10	4.29	9.88	60	3.05	0.903	3.45	0.578	0.755	9.85
	16†	4.87	9.93	60	2.99	0.784	3.45	0.665	0.739	9.91
	seq.	1	1.00	0	4.04	0.522	3.04	1	1	1
2048	1	0.980	1.00	0	4.04	0.563	2.96	0.982	0.998	1.00
	2	1.39	2.00	0	3.92	0.772	2.96	0.716	0.970	2.00
	4	2.34	4.00	0	3.34	0.781	2.96	0.707	0.827	4.00
	6	2.95	5.99	33	3.19	0.886	3.41	0.624	0.790	5.99
	8	3.45	7.98	50	3.03	0.957	3.41	0.577	0.750	7.97
	10	3.53	9.93	60	2.97	1.14	3.41	0.484	0.735	9.93
	16†	3.44	9.95	60	3.00	1.19	3.41	0.466	0.742	9.94
	seq.	1	1.00	0	4.04	0.553	2.96	1	1	1
4096	1	0.981	1.00	0	4.04	0.780	2.92	0.981	1.00	1.000
	2	1.61	2.00	0	3.63	0.851	2.92	0.898	0.898	2.00
	4	2.79	4.00	0	3.14	0.852	2.92	0.898	0.777	4.00
	6	3.11	5.99	33	3.06	1.11	2.92	0.686	0.756	5.99
	8	3.65	7.99	50	2.95	1.22	2.92	0.627	0.729	7.99
	10	2.98	9.91	60	3.00	1.89	3.39	0.405	0.743	9.91
	16†	2.85	9.94	60	2.96	1.95	3.40	0.391	0.732	9.93
	seq.	1	1.00	0	4.04	0.764	2.92	1	1	1

† oversubscribed: more worker threads than the 10 logical processors.

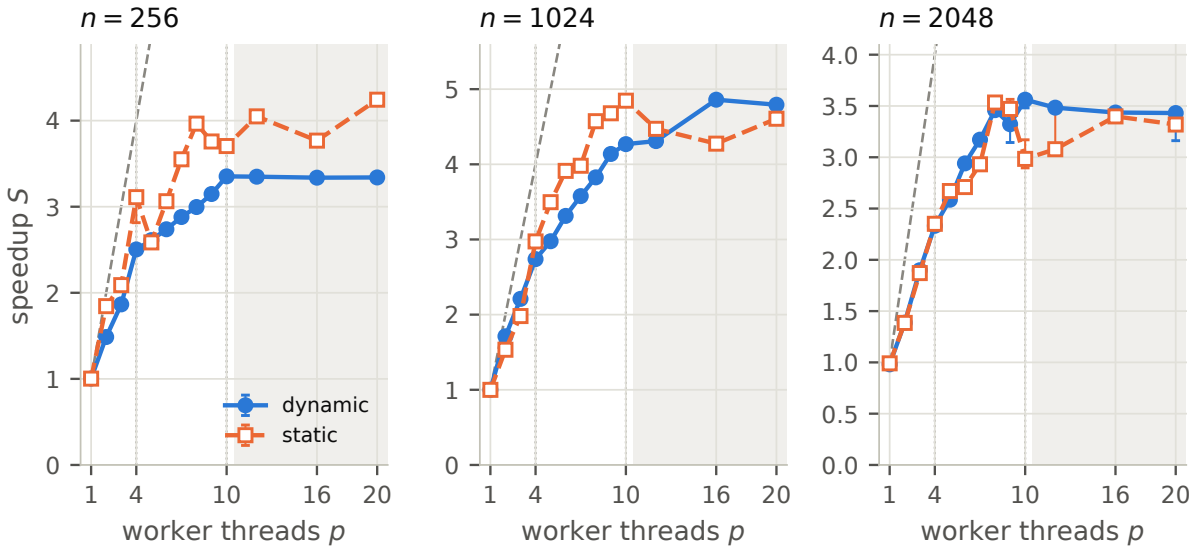


Figure 5: Speedup of the static (contiguous bands) and dynamic (row by row) schedules at $n = 256$, 1024 and 2048, with interval bars; dashed: $S = p$.

Each multiply-add reads 8 bytes of B, so a multiplication that takes T requests B at $8n^3/T$ bytes per second from the memory hierarchy. At $n = 4096$, where B occupies 128 MiB, the sequential kernel requested 42 GB/s. From $p = 4$ on, the parallel kernel requested 103 to 154 GB/s, and 120 GB/s with 12, 16 and 20 threads. This is the memory bandwidth Apple specifies for the M4 [16]. If every byte of B came from memory, the speedup at $n = 4096$ could not exceed $120/42 \approx 2.8$. The measured speedups for $p \geq 4$ were 2.4 to 3.6, and 2.8 for every oversubscribed p. At $n = 2048$ ($B = 32$ MiB), 10 threads requested 209 GB/s, more than the memory can supply, so part of B came from the caches.

4.6 Static against dynamic rows

Contrary to H4, the static schedule was faster than the dynamic one at $n = 256$ and 1024 for most thread counts above four (Fig. 5, Table 7). For $5 < p \leq 10$ it was faster by 11 to 32% at $n = 256$ and by 11 to 20% at $n = 1024$. At $p = 5$ it was 1% slower at $n = 256$ and 18% faster at $n = 1024$. Only at $n = 2048$ did static lose, by 8 to 16% at $p = 6$, 7 and 10, while it was within 5% at the other thread counts up to 10. The static runs did leave cores idle, as H4 expected. At $p = 10$ they kept 5.8 ($n = 256$), 8.6 (1024) and 9.1 (2048) cores busy, against 9.3 to 9.9 for the dynamic schedule. The E cores also did less of the work: 24 to 38% at $p = 10$, not the 60% of the rows that six of the ten bands hold. Yet the static runs needed far fewer cycles per multiply-add for the same instructions: 0.65 against 1.18 at $n = 256$, and 0.69 against 0.90 at $n = 1024$ ($p = 10$). The excess of the dynamic schedule shrank as n grew: 0.53, 0.22 and -0.09 cycles per multiply-add at $n = 256$, 1024 and 2048 ($p = 10$). It was present with four threads too (0.16 at $n = 256$), when every thread ran on a P core.

4.7 Oversubscription

Beyond 10 threads, the dynamic schedule gained nothing at $n = 256$ and 512 (within 0.5% of $p = 10$). It lost up to 5.0% at $n = 64$ and at $n = 2048$ and 4096. At $n = 1024$, 16 and 20 threads were 14 and 12% faster than 10, with fewer cycles per multiply-add (0.78 and 0.80 against 0.90) for the same busy cores and E-core share. At $n = 128$, 20 threads were 6% faster than 10. One process per configuration and no cache-miss counters cannot tell why.

Table 7: Speedup of par_ikj with the dynamic and the static schedule, and busy cores U of the static runs.

	$n = 256$			$n = 1024$			$n = 2048$		
p	dyn.	static	busy	dyn.	static	busy	dyn.	static	busy
1	1.00	1.00	1.0	1.00	0.999	1.0	0.981	0.991	1.0
2	1.49	1.84	2.0	1.71	1.53	2.0	1.39	1.38	2.0
3	1.87	2.09	3.0	2.21	1.98	3.0	1.90	1.87	3.0
4	2.50	3.11	3.9	2.74	2.97	4.0	2.33	2.35	3.9
5	2.61	2.59	3.5	2.98	3.50	4.8	2.59	2.67	5.0
6	2.74	3.06	4.5	3.31	3.91	5.8	2.94	2.71	5.9
7	2.88	3.55	5.3	3.58	3.98	6.2	3.17	2.93	6.9
8	3.00	3.97	6.2	3.83	4.57	7.5	3.46	3.53	7.9
9	3.15	3.76	6.0	4.14	4.68	7.9	3.32	3.47	8.8
10	3.35	3.70	5.8	4.27	4.85	8.6	3.56	2.98	9.1
12†	3.35	4.05	6.7	4.31	4.47	7.6	3.48	3.08	8.6
16†	3.34	3.77	6.1	4.86	4.27	8.2	3.44	3.40	9.6
20†	3.34	4.24	7.4	4.79	4.61	9.2	3.43	3.32	9.5

† oversubscribed: more worker threads than the 10 logical processors.

Table 8: Peak RSS (MiB) of ikj and par_ikj (dynamic), Java heap allocated over the whole process by par_ikj (MiB), and element storage $24n^2$ (MiB).

		par_ikj, peak RSS				allocated		
n	ikj	$p = 1$	$p = 4$	$p = 10$	$p = 20$	$p = 1$	$p = 20$	$24n^2$
64	62.8	83.0	106	110	168	23.5	134	0.0938
128	63.6	68.7	77.0	88.4	104	9.18	41.8	0.375
256	66.0	67.7	70.0	73.6	78.2	9.29	16.2	1.50
512	70.6	71.9	73.2	75.5	78.8	13.7	17.1	6.00
1024	88.8	90.0	91.0	93.4	119	31.7	34.4	24.0
2048	162	163	164	166	169	104	106	96.0
4096	551	554	554	556	559	392	395	384

4.8 Memory

At $n \geq 2048$ the peak RSS of par_ikj exceeded that of ikj by 1 to 8 MiB for every p (Table 8). Between $p = 1$ and 20 it grew from 163 to 169 MiB at $n = 2048$ and from 554 to 559 MiB at $n = 4096$, where it was 1.4 times the $24n^2$ bytes of the three matrices. At small n it grew with p. At $n = 64$ it rose from 83 MiB ($p = 1$) to 168 MiB ($p = 20$), against 63 MiB for ikj. The heap allocation explains this. At $n = 64$, par_ikj allocated 23.5 MiB of Java objects over the whole process with one thread and 134.5 MiB with 20. Each call builds p task objects and their futures, a process makes tens of thousands of calls at this size, and the garbage collector ran at most once per process at $n \leq 128$, so the garbage stayed in the heap. At $n = 1024$, 16 and 20 threads raised the peak RSS by 28 to 31 MiB without allocating much more (34.4 MiB at $p = 20$ against 31.7 MiB at $p = 1$); this was not explained.

4.9 Vectorisation

The decoded machine code confirms the vectorisation (Table 9). With SuperWord, the compiled kernels of ikj and of par_ikj (`ikjRow`, and the task lambda into which it is inlined) contain 8 `fmul.2d` and 8 `fadd.2d` instructions, an inner loop unrolled over 16 elements, two per instruction. They also contain two scalar multiplies and two scalar adds for the iterations outside the vector loop. Without SuperWord they contain only scalar multiplies and adds, 10 of each. No fused multiply-add appears in either.

Table 9: Double-precision floating-point instructions in the final C2-compiled kernels with SuperWord on and off (`results/simd`): two-lane vector and scalar multiplies and adds, fused multiply-adds, and 128-bit loads/stores.

run	compiled method	SuperWord	fmul.2d	fadd.2d	fmul	fadd	fused	q ld/st
ikj	`DenseMatMul::ikj`	on	8	8	2	2	0	16/24
ikj	`DenseMatMul::ikj`	off	0	0	10	10	0	0/0
par_ikj	`ParallelMatMul::ikjRow`	on	8	8	2	2	0	16/24
par_ikj	`ParallelMatMul::lambda$ikj$1`	on	8	8	2	2	0	16/24
par_ikj	`ParallelMatMul::ikjRow`	off	0	0	10	10	0	0/0
par_ikj	`ParallelMatMul::lambda$ikj$1`	off	0	0	10	10	0	0/0

Table 10: SuperWord on against off: median time per multiplication, the ratio off/on, and the instructions and cycles per multiply-add (all cores).

		time per multiplication			instr./mult.-add		cycles/mult.-add	
n	configuration	on (ms)	off (ms)	off/on	on	off	on	off
512	ikj (seq.)	17.4	25.5	1.46	3.21	5.90	0.528	0.777
512	par_ikj, $p = 4$	7.29	8.48	1.16	3.21	5.89	0.795	0.872
512	par_ikj, $p = 10$	4.38	6.01	1.37	3.52	5.89	1.01	1.36
1024	ikj (seq.)	138	208	1.51	3.04	5.83	0.522	0.785
1024	par_ikj, $p = 4$	50.5	72.2	1.43	3.04	5.82	0.659	0.929
1024	par_ikj, $p = 10$	32.4	45.3	1.40	3.45	5.82	0.903	1.27
2048	ikj (seq.)	1 173	1 754	1.50	2.96	5.79	0.553	0.820
2048	par_ikj, $p = 4$	503	685	1.36	2.96	5.78	0.781	1.04
2048	par_ikj, $p = 10$	329	433	1.32	3.41	5.79	1.14	1.47

Without SuperWord (Table 10, Fig. 6), the instructions per multiply-add rose from 3.0–3.5 to 5.8–5.9, by a factor of 1.7 to 2.0. The kernel became 1.46 to 1.51 times slower sequentially, 1.16 to 1.43 times with four threads and 1.32 to 1.40 times with ten. At $n = 1024$ and 2048 the gain from vectorisation fell as p grew, from 1.51 and 1.50 at $p = 1$ to 1.40 and 1.32 at $p = 10$. At $n = 512$ the smallest gain was at $p = 4$ (1.16). There, the SuperWord-on run needed 0.80 cycles per multiply-add, 50% more than the sequential one.

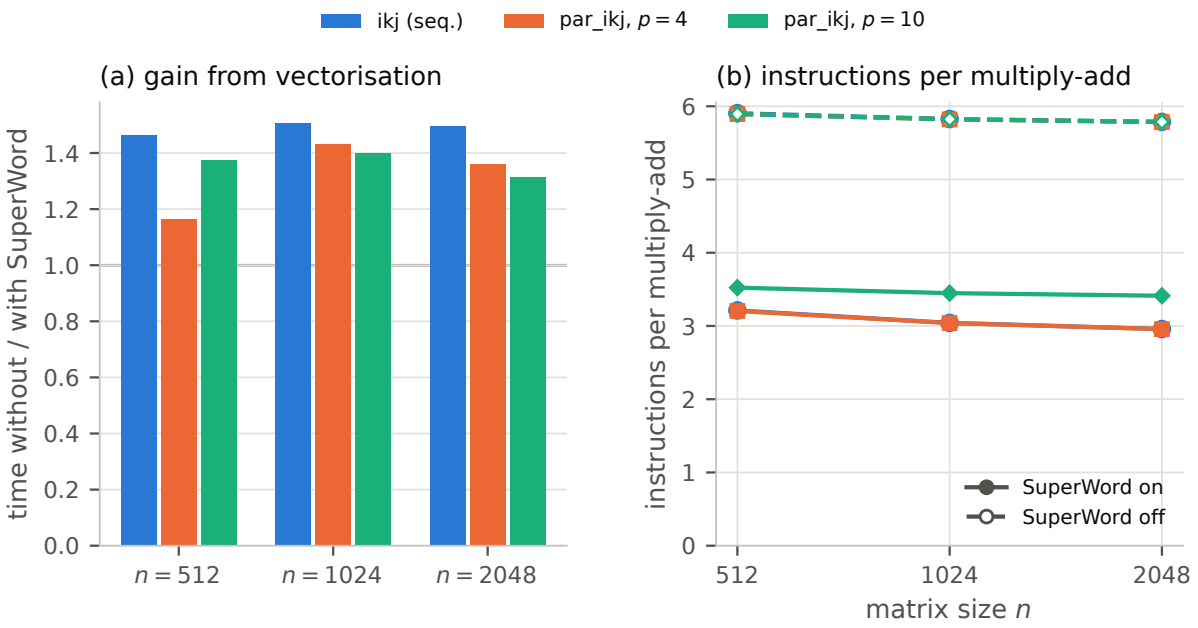


Figure 6: (a) Time without SuperWord divided by the time with it (above 1: vectorisation helps). (b) Instructions per multiply-add with SuperWord on (filled) and off (hollow, dashed); the lines of ikj and of par_ikj with $p = 4$ coincide.

5 Discussion

Answer to the research question. On this machine, parallel execution of the i-k-j kernel helped from $n = 128$ on, where one multiplication lasts 0.3 ms or more. Four threads, one per P core, gave a speedup of 2.3 to 2.8 at every such size. Beyond four threads, the gain depended on where the data lived. While B fitted in the caches ($n = 256$ to 1024), the E cores raised the speedup to 3.4 to 4.3 at $p = 10$. At $n = 4096$, where B streams from memory, the memory bandwidth held it near 3. At $n = 64$ (30 μs) waking the workers cost as much as the work, and parallel execution did not help. The fastest thread count was 10 or more for $n = 256$ to 2048. At $n = 4096$ it was 8, but its speedup of 3.63 is one process in a range of 2.4 to 3.6 for $p \geq 4$. The efficiency never exceeded 0.86 with two or more threads, and it was 0.30 to 0.43 at $p = 10$.

H1: identical results. Supported. Every configuration agreed bit for bit with every other, in the verification and in all 835 measured checksums. Each thread computes whole rows with the same operations in the same order as ikj, and vectorisation does not change the order within an entry of C. Without fused multiply-add [12], the rounding is the same.

H2: the cost of one thread. Partly supported. At $n = 64$ the overhead was 24% (predicted: above 10%), and from $n = 256$ to 1024 it was within $\pm 0.4\%$ (predicted: below 1% from $n = 512$). The cost per call, 7.2 μs, was smaller than the predicted tens of microseconds. The prediction failed at $n = 2048$ and 4096, where the one-thread version was 1.9 and 2.1% slower. There it ran the same instructions with about 2% more cycles, which points to the memory system rather than to the hand-off: the code of `ikjRow` is compiled separately from `ikj`, and the worker is a different thread from the main one. The session cannot separate these causes. At $n = 4096$, moreover, the two ikj controls differed by 1.2%, so part of the gap is variation between processes.

H3: small matrices. Partly supported. At $n = 64$ the speedup stayed below 2 (at most 1.15). It fell below 1 with one to four threads, not at high p as predicted: with many threads, the extra workers mostly find no rows left (2.9 busy cores at $p = 10$). This is Amdahl's bound [1] with the

hand-off as the serial part. At $n = 128$ the hypothesis failed: the speedup reached 2.5 to 2.7. A multiplication of 0.3 ms is long enough for the four P cores, which took more than 90% of the rows before the E-core threads joined in.

H4: P and E cores, static and dynamic. Partly supported. The first part held. The gain per added thread was 0.44 to 0.59 up to $p = 4$ and 0.03 to 0.26 from 4 to 10 ($n \geq 256$). The E-core share was near zero up to $p = 4$ and then followed $(p - 4)/p$, and $S(10)$ was well below 10 (at most 4.27). The 4P+6E capacity line of Fig. 2, $4 + 6r = 7.6$ at $p = 10$, is the speedup that core types alone would allow if every thread ran as fast as a sequential P core. The measured 4.0 at $n = 512$ is about half of it, because the clock fell and contention added cycles; the asymmetric chip of Hill and Marty [2] is only part of the story.

The second part failed: the static schedule was faster than the dynamic one at $n = 256$ and 1024. Two observations explain why the slowest core did not set the time. First, the static bands did not stay on the cores where they started. The E cores did 24 to 38% of the work, not the 60% of the rows that six of the ten bands hold, so macOS moved the remaining bands to the P cores once those had finished their own. Apple's warning about static distribution [6] assumes threads that stay on their cores. Second, the dynamic schedule paid extra cycles per multiply-add, and the excess fell roughly as $1/n$, from 0.53 at $n = 256$ to 0.22 at 1024 and none at 2048. The most likely cause is false sharing. Under the dynamic schedule, consecutive rows go to different threads and are computed at the same time. The rows of C are separate arrays of $8n + 16$ bytes, allocated one after another, so the end of row i and the start of row $i + 1$ normally share a 128-byte cache line. Both threads write that line at each of the n steps of k, so it moves between cores up to $2n$ times per row while the row performs n^2 multiply-adds: a cost per multiply-add proportional to $1/n$. The static bands share only $p - 1$ boundaries. The counters do not measure coherence traffic, so this is an inference. A dynamic schedule with chunks of several rows, or rows padded to whole cache lines, would test it.

H5: contention for caches and memory. Supported. The cycles per multiply-add rose with p for every $n \geq 256$. With four threads, all on P cores and running the same instructions, they were already 1.11 to 1.50 times the sequential ones. At $p = 10$ they were 2.07 and 2.47 times the sequential ones at $n = 2048$ and 4096, against 1.73 to 1.91 at $n = 512$ and 1024. The efficiency at $p = 10$ was correspondingly lower at $n \geq 2048$ (0.36 and 0.30) than at 512 and 1024 (0.40 and 0.43). No speedup was superlinear. The largest effect appeared where B fits no cache. At $n = 4096$ the kernel requested B at about the 120 GB/s memory bandwidth of the chip from $p = 4$ on, the roofline limit of a kernel with one multiply-add per 8 bytes [3], and the speedup for 12 to 20 threads, 2.8, equals the ratio of that bandwidth to the sequential demand to two digits. Speedups above 2.8 at $n = 4096$ (up to 3.6) require threads to share lines of B in the caches, which happens when they work on nearby rows at the same time. This sharing is not controlled, which may explain why the speedup at $n = 4096$ jumped between 2.4 and 3.6 from one thread count to the next instead of following p.

H6: oversubscription. Mostly supported. Twelve to twenty threads cost at most 5% against ten, within the predicted bound, and gave no further speedup at five of the seven sizes. The exceptions were $n = 1024$ (12 to 14% faster with 16 and 20 threads, with fewer cycles per multiply-add) and $n = 128$ (6% with 20 threads). They are single processes and unexplained.

H7: memory. Partly refuted. At $n \geq 2048$ the peak RSS did not depend on p beyond 8 MiB, as predicted. At $n \leq 128$ it grew by up to 85 MiB. This is not data shared by the threads but garbage. `invokeAll` allocates p task objects and futures on every call, small calls are many, and with an 8 GiB heap limit the collector rarely ran. In a long-running service the same garbage would cost collections instead of resident memory. A reusable array of tasks would remove it. The 28 to 31 MiB increase at $n = 1024$ with 16 and 20 threads remains unexplained.

H8: vectorisation. Mostly supported. Without SuperWord the code was scalar, and the instructions per multiply-add rose by a factor of 1.7 to 2.0, close to the predicted doubling. The slowdown was largest with one thread (1.46 to 1.51) and smaller at $p = 10$, $n = 2048$ (1.32), as predicted. More threads leave less to gain from executing fewer instructions, because the cores spend more of their cycles waiting for data. The smallest gain, however, was at $p = 4$, $n = 512$ (1.16), which the hypothesis did not predict. The SuperWord-on run there needed 50% more cycles per multiply-add than the sequential one, which suggests that the four P cores were already limited by their shared L2 cache.

Limitations.

- **One machine, one session, one process per configuration.** The IQRs describe variation within a process. Between processes, two runs of the same configuration differed by up to 7.4% (ikj, $n = 128$) and 15% (par_ikj, $p = 10$, $n = 2048$). Comparisons between neighbouring thread counts are therefore uncertain by more than their error bars, especially at $n \geq 2048$ and for $p \geq 4$.
- **No affinity.** macOS chose the cores. The measured placement (four threads on P cores) is an observation of its scheduler on an idle machine, not a controlled factor.
- **Counters, not miss events.** The counters are sums over the whole process; cache misses and coherence traffic were not measured. The explanations of the schedule result (false sharing) and of the $n = 4096$ limit (memory bandwidth) are inferences from cycles, instructions and the requested traffic, not measurements of traffic.
- **JIT variation.** The instructions per multiply-add of par_ikj took different values in different processes of the same size, for example 2.96 and 3.41 at $n = 2048$, so the JIT did not always compile the kernel the same way. Single-process comparisons include this variation.
- **Heat.** The fanless machine still throttled at the end of the $n = 4096$ block (2.59 GHz in the last control run), despite the cool-down pauses. Its effect on the time was small (2.5%) because that kernel waits for memory.
- **Background CPU.** Other processes used a median of 0.06 cores. For the session, this figure relies on the per-QoS CPU times instead of `/usr/bin/time` (Section 3.7), a substitution checked on the pilot and smoke sessions.

6 Conclusions

For the tested kernel and machine, this study establishes the following:

1. A row-parallel i-k-j multiplication on a fixed thread pool is bitwise identical to the sequential kernel for any thread count and schedule, with or without vectorisation. Rows written by one thread each, and the happens-before order of `invokeAll`, are enough for safe sharing.
2. Parallel execution helps from $n = 128$ on. Four threads give a speedup of 2.3 to 2.8, and all ten cores give 3.0 to 4.3 for $n \geq 256$, an efficiency of 0.30 to 0.43. At $n = 64$ it does not help: the hand-off costs about 7 μs per call, and the one-thread version is 24% slower.
3. macOS runs four threads on the P cores and every further thread on an E core, which retires about half to two thirds of the instructions per second of a P core. Together with the clock falling from 4.0 to 3.0–3.8 GHz as cores are added, this explains why ten threads are about four times faster, not ten.
4. Threads contend for the shared caches and memory: with four threads the cycles per multiply-add rise by 11 to 50%. At $n = 4096$ the kernel reaches the 120 GB/s memory bandwidth of the chip from about four threads, and its speedup stays near 3.

5. More threads than logical processors cost at most 5% and gain only in isolated cases.
6. A static assignment of row bands is up to 32% faster than a row-by-row dynamic one at $n = 256$ and 1024, despite the heterogeneous cores, most likely because the dynamic schedule makes adjacent rows share cache lines between threads. The dynamic schedule wins only at $n = 2048$, by 8 to 16% at three of the thread counts.
7. SIMD vectorisation by the JIT (two doubles per instruction) cuts the instructions per multiply-add by 40 to 49% and makes the kernel 1.16 to 1.51 times faster; it gains less when more threads compete for memory.
8. Peak memory is independent of p for large matrices. For small ones it grows with p because every call allocates task objects.

Four questions remain open. The first is whether false sharing explains the cost of the dynamic schedule; chunked or padded rows would test it. The second is why 16 and 20 threads beat 10 at $n = 1024$. The third is why the one-thread version is 2% slower at $n \geq 2048$. The fourth is how much the threads share B in the caches at $n = 4096$. Cache-miss and coherence counters, and several processes per configuration, would answer them.

7 References and materials

Reproducibility materials. All code, data and scripts are in the directory `assignment3/` of the GitHub repository below, at the tag `assignment3-v1.0`:

`https://github.com/Xami650/Individual-Assignments-Big-Data-ULPGC`

Its README gives the exact build and run commands and the software versions. The materials are:

- `src/`: the Java methods, the process counters, the benchmark driver and the JUnit tests; `pom.xml`: the Maven build.
- `config/bench.json`: the declared protocol, read by every script.
- `scripts/`: input generation (`gen_inputs.py`), correctness verification (`verify.py`), the benchmark harness (`run_benchmarks.py`), the disassembly of the compiled kernels (`simd_evidence.py`) and the environment snapshot.
- `data/checksums.sha256`: the digests of the inputs, which are regenerated in a few seconds instead of being stored.
- `results/correctness.csv`: the outcome of the correctness checks; `results/simd/`: the decoded machine code of the kernels and its instruction counts.
- `results/raw/20261104-030623/`: every raw sample, warm-ups included (`samples.csv`), one row per process with status, settings, JVM activity, peak RSS and background CPU (`runs.csv`), the environment snapshot (`env.txt`) and the protocol used (`config.json`).
- `analysis/analyze.py`: the script that produces every table and figure in this report from the raw data.

Reuse of Assignments 1 and 2. The baseline kernel, the input generator and the input files, the correctness tolerance, and the structure of the benchmark driver, the harness, the verification script and the analysis script come from Assignment 2 at the tag `assignment2-v1.0`, which took the generator, the tolerance and the harness from Assignment 1. The reused files are adapted copies, and each one names its origin.

Use of generative AI. Claude Code (Anthropic, model Claude Opus 5.5) was used as a programming and writing assistant. It helped plan the study, draft and debug the Java code, tests, scripts and analysis, find the macOS interfaces for the per-core counters, diagnose the thermal behaviour found in the pilot, and draft and revise the text of this report. All measurements were produced by running the code on the author's machine, and every result in Section 4 is computed from the committed data by the scripts in the repository. Each cited source was checked against the original publication or documentation. The author reviewed the code and the text and is responsible for the work and its conclusions.

References

[1] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conf.*, vol. 30, Atlantic City, NJ, USA, Apr. 1967, pp. 483–485.

[2] M. D. Hill and M. R. Marty, "Amdahl's law in the multicore era," *Computer*, vol. 41, no. 7, pp. 33–38, Jul. 2008, doi: 10.1109/MC.2008.209.

[3] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009, doi: 10.1145/1498765.1498785.

[4] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2019, ch. 1.

[5] OpenMP Architecture Review Board, "OpenMP application programming interface, version 5.2," Nov. 2021. [Online]. Available: `https://www.openmp.org/spec-html/5.2/openmp.html`

[6] Apple Inc., "Tuning your code's performance for Apple silicon," Apple Developer Documentation. Accessed: Oct. 29, 2026. [Online]. Available: `https://developer.apple.com/documentation/apple-silicon/tuning-your-code-s-performance-for-apple-silicon`

[7] J. Gosling, B. Joy, G. Steele, G. Bracha, A. Buckley, D. Smith, and G. Bierman, *The Java Language Specification, Java SE 25 Edition*. Oracle America, Inc., 2025, ch. 17. [Online]. Available: `https://docs.oracle.com/javase/specs/jls/se25/html/jls-17.html`

[8] Oracle, "Package java.util.concurrent," *Java SE 25 & JDK 25 API Specification*, section "Memory Consistency Properties." Accessed: Oct. 29, 2026. [Online]. Available: `https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/package-summary.html`

[9] B. Goetz, T. Peierls, J. Bloch, J. Bowbeer, D. Holmes, and D. Lea, *Java Concurrency in Practice*. Upper Saddle River, NJ, USA: Addison-Wesley, 2006, ch. 8.

[10] Apple Inc., `<sys/resource.h>` (`struct rusage_info_v6`) and `<libproc.h>` (`proc_pid_rusage`, `proc_pidinfo`), macOS SDK of the Command Line Tools for Xcode, header files.

[11] S. Larsen and S. Amarasinghe, "Exploiting superword level parallelism with multimedia instruction sets," in *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2000, pp. 145–156, doi: 10.1145/349299.349320.

[12] J. Darcy, "JEP 306: Restore always-strict floating-point semantics," OpenJDK, delivered in JDK 17. Accessed: Oct. 29, 2026. [Online]. Available: `https://openjdk.org/jeps/306`

[13] M. Cimadamore, "JEP 454: Foreign function & memory API," OpenJDK, delivered in JDK 22. Accessed: Oct. 29, 2026. [Online]. Available: `https://openjdk.org/jeps/454`

[14] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd Annu. ACM SIGPLAN Conf. Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, 2007, pp. 57–76, doi: 10.1145/1297027.1297033.

[15] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA, USA: SIAM, 2002.

[16] Apple Inc., "MacBook Air (15-inch, M4, 2025) – Tech Specs," Apple Support. Accessed: Nov. 4, 2026. [Online]. Available: `https://support.apple.com/en-us/122210`

[17] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020, doi: 10.1038/s41586-020-2649-2.